Student Database Management System Using Hashing

Name	NAMRATA PRABHAKAR RAUT
Roll No.	SEB COMP 54
Batch	S6
Academic Year	2026-2027

Project Report

1. Problem Statement

To develop a GUI-based Student Database Management System using the Data Structure and Algorithm concept of Hashing (Hash Table with Separate Chaining) for fast insertion, searching, updating and deletion of student records.

2. Algorithm

A. Algorithm for Hash Function (Division Method)

1.Start.
2.Take the Roll Number of the student as the key.
3.Divide the key by the table size (TABLE_SIZE = 10).
4.The remainder is the index (bucket number): index = key % TABLE_SIZE.
5.Return the index.
6.Stop.

B. Algorithm for Adding Student (Insertion)

1.Start.
2.Enter Roll Number, Name, Department and Marks.
3.Calculate the bucket index using the hash function.
4.Traverse the chain (list) stored at that index.
5.If a student with the same Roll Number already exists, display "Roll number already exists" and stop.
6.Otherwise, append the new student record at the end of the chain (collision handled by chaining).
7.Display confirmation message.
8.Stop.

C. Algorithm for Searching Student

1.Start.
2.Enter the Roll Number to search.
3.Calculate the bucket index using the hash function.
4.Go directly to that bucket.
5.Compare the Roll Number with each record in the chain of that bucket.
6.If both are equal, display the student details.
7.If the chain ends without a match, display "Student Not Found".
8.Stop.

D. Algorithm for Updating Student

1.Start.
2.Enter the Roll Number and the new Name, Department and Marks.
3.Search the student using the searching algorithm.
4.If the student is found, replace the old details with the new details.
5.Display "Record Updated" message.
6.If not found, display "Student Not Found".
7.Stop.

E. Algorithm for Deleting Student

1.Start.
2.Enter the Roll Number to delete.
3.Calculate the bucket index using the hash function.
4.Search the chain of that bucket for the Roll Number.
5.If found, remove the record from the chain.
6.Display "Record Deleted" message.
7.If not found, display "Student Not Found".
8.Stop.

3. Program

```
import tkinter as tk
from tkinter import ttk, messagebox

TABLE_SIZE = 10
```

```
hash_table = [[] for _ in range(TABLE_SIZE)]

# Hash Function (Division Method)
def hash_function(roll_no):
    return roll_no % TABLE_SIZE

# Insert Student (Separate Chaining)
def insert_student(student):
    index = hash_function(student["roll"])
    for s in hash_table[index]:
        if s["roll"] == student["roll"]:
            return False
    hash_table[index].append(student)
    return True

# Search Student
def search_student(roll):
    index = hash_function(roll)
    for s in hash_table[index]:
        if s["roll"] == roll:
            return s
    return None

# Delete Student
def delete_student(roll):
    index = hash_function(roll)
    for s in hash_table[index]:
        if s["roll"] == roll:
            hash_table[index].remove(s)
            return True
    return False

# Read and Validate Input Fields
def read_fields():
    roll = int(roll_entry.get())
    name = name_entry.get().strip()
    dept = dept_entry.get().strip()
    marks = float(marks_entry.get())
    if name == "" or dept == "":
        raise ValueError
    return {"roll": roll, "name": name, "dept": dept, "marks": marks}

# Clear Entry Fields
def clear_entries():
    roll_entry.delete(0, tk.END)
    name_entry.delete(0, tk.END)
    dept_entry.delete(0, tk.END)
    marks_entry.delete(0, tk.END)

# Display All Students
def display_all():
    for row in tree.get_children():
        tree.delete(row)
    count = 0
    for index in range(TABLE_SIZE):
        for s in hash_table[index]:
            tree.insert("", tk.END, values=(
                index, s["roll"], s["name"], s["dept"], s["marks"]))
            count += 1
    count_label.config(text="Total Students: " + str(count))

# Show Hash Table (Bucket View)
def show_buckets():
    bucket_view.delete("1.0", tk.END)
```

```
    for index in range(TABLE_SIZE):
        line = "Bucket " + str(index) + " : "
        if not hash_table[index]:
            line += "Empty"
        else:
            line += " -> ".join(
                str(s["roll"]) + " (" + s["name"] + ")"
                for s in hash_table[index])
        bucket_view.insert(tk.END, line + "\n")

# Add Student
def add_student():
    try:
        student = read_fields()
        if insert_student(student):
            messagebox.showinfo("Success", "Student added successfully!")
            clear_entries()
            display_all()
            show_buckets()
        else:
            messagebox.showwarning("Duplicate", "Roll number already exists.")
    except ValueError:
        messagebox.showerror("Error", "Please enter valid details.")

# Search Student
def search_gui():
    try:
        roll = int(roll_entry.get())
        s = search_student(roll)
        if s:
            for row in tree.get_children():
                tree.delete(row)
            tree.insert("", tk.END, values=(
                hash_function(roll), s["roll"], s["name"],
                s["dept"], s["marks"]))
            messagebox.showinfo("Found", "Student found in bucket "
                                + str(hash_function(roll)))
        else:
            messagebox.showwarning("Not Found", "Student not found.")
    except ValueError:
        messagebox.showerror("Error", "Enter a valid Roll Number.")

# Update Student
def update_gui():
    try:
        student = read_fields()
        s = search_student(student["roll"])
        if s:
            s.update(student)
            messagebox.showinfo("Success", "Student record updated!")
            clear_entries()
            display_all()
            show_buckets()
        else:
            messagebox.showwarning("Not Found", "Student not found.")
    except ValueError:
        messagebox.showerror("Error", "Please enter valid details.")

# Delete Student
def delete_gui():
    try:
        roll = int(roll_entry.get())
        if delete_student(roll):
            messagebox.showinfo("Deleted", "Student record deleted.")
```

```
            clear_entries()
            display_all()
            show_buckets()
        else:
            messagebox.showwarning("Not Found", "Student not found.")
    except ValueError:
        messagebox.showerror("Error", "Enter a valid Roll Number.")

# GUI
root = tk.Tk()
root.title("Student Database Management System")
root.geometry("850x720")
root.configure(bg="#f4f4f4")

title = tk.Label(root, text="STUDENT DATABASE MANAGEMENT SYSTEM",
                 font=("Arial", 18, "bold"), bg="#337ab7", fg="white")
title.pack(fill=tk.X, pady=10)

# Input Fields
frame = tk.Frame(root, bg="#f4f4f4")
frame.pack(pady=10)

labels = ["Roll No", "Name", "Department", "Marks"]
entries = []
for i, text in enumerate(labels):
    tk.Label(frame, text=text + ":", bg="#f4f4f4").grid(
        row=i // 2, column=(i % 2) * 2, padx=8, pady=5, sticky="e")
    e = tk.Entry(frame, width=22)
    e.grid(row=i // 2, column=(i % 2) * 2 + 1, padx=8, pady=5)
    entries.append(e)
roll_entry, name_entry, dept_entry, marks_entry = entries

# Buttons
btn_frame = tk.Frame(root, bg="#f4f4f4")
btn_frame.pack(pady=5)

tk.Button(btn_frame, text="Add", width=10, bg="#28a745", fg="white",
          command=add_student).pack(side=tk.LEFT, padx=4)
tk.Button(btn_frame, text="Search", width=10, bg="#17a2b8", fg="white",
          command=search_gui).pack(side=tk.LEFT, padx=4)
tk.Button(btn_frame, text="Update", width=10, bg="#ffc107",
          command=update_gui).pack(side=tk.LEFT, padx=4)
tk.Button(btn_frame, text="Delete", width=10, bg="#dc3545", fg="white",
          command=delete_gui).pack(side=tk.LEFT, padx=4)
tk.Button(btn_frame, text="Display All", width=10, bg="#6f42c1", fg="white",
          command=display_all).pack(side=tk.LEFT, padx=4)
tk.Button(btn_frame, text="Clear", width=10,
          command=clear_entries).pack(side=tk.LEFT, padx=4)

# Student Table
columns = ("Bucket", "Roll No", "Name", "Department", "Marks")
tree = ttk.Treeview(root, columns=columns, show="headings", height=10)
for col in columns:
    tree.heading(col, text=col)
    tree.column(col, width=140, anchor="center")
tree.pack(padx=10, pady=10)

count_label = tk.Label(root, text="Total Students: 0", bg="#f4f4f4")
count_label.pack()

# Hash Table Bucket View
tk.Label(root, text="Hash Table (Bucket View)", bg="#f4f4f4",
         font=("Arial", 11, "bold")).pack()
bucket_view = tk.Text(root, height=11, width=95)
```

```
bucket_view.pack(pady=5)

# Sample Records
sample = [
    (101, "Amit Patil", "Computer", 82.5),
    (102, "Neha Joshi", "IT", 76.0),
    (111, "Rahul Deshmukh", "Computer", 68.5),
    (205, "Sneha Kulkarni", "ENTC", 91.0),
    (215, "Pooja More", "IT", 73.5),
    (308, "Kiran Shinde", "Mechanical", 64.0),
]
for r, n, d, m in sample:
    insert_student({"roll": r, "name": n, "dept": d, "marks": m})

display_all()
show_buckets()
root.mainloop()
```

4. Output

The application creates a Student Database Management System and displays a GUI window containing:

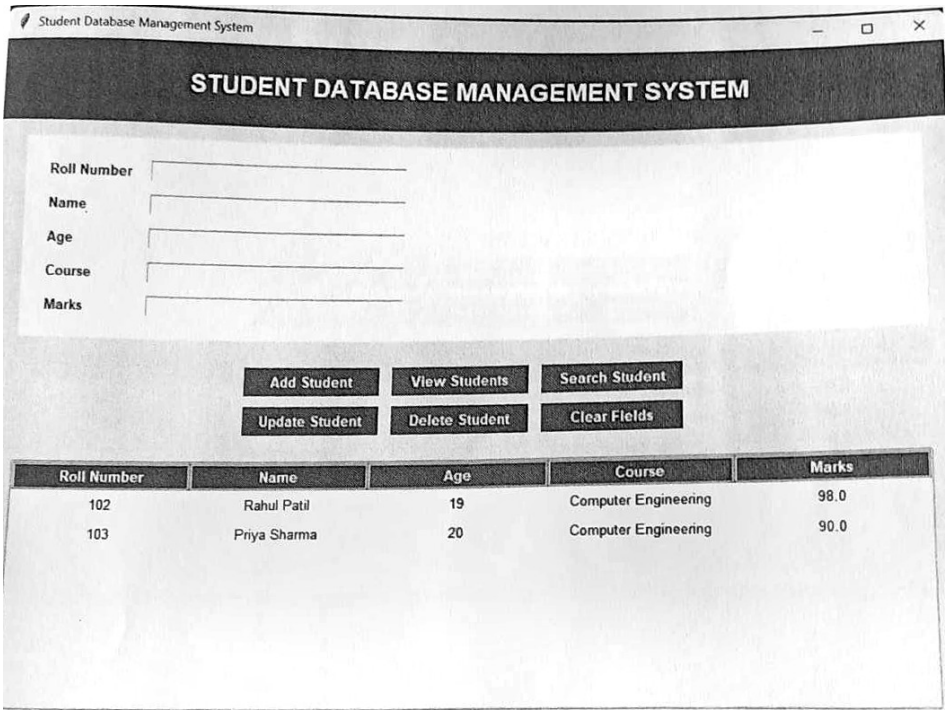


Figure 1: Main window of the Student Database Management System.

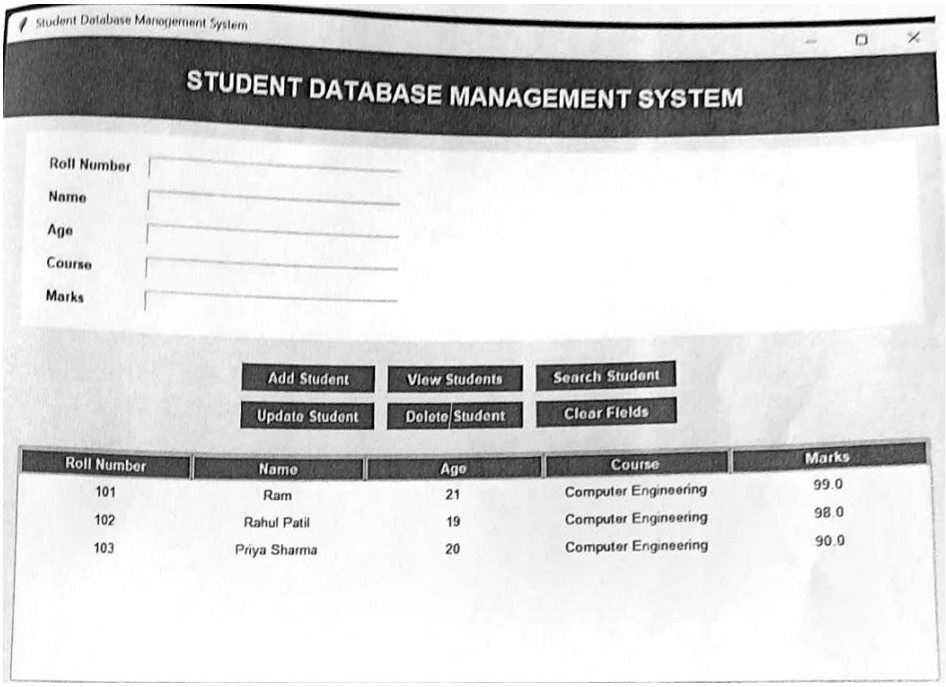


Figure 2: Displaying student records.

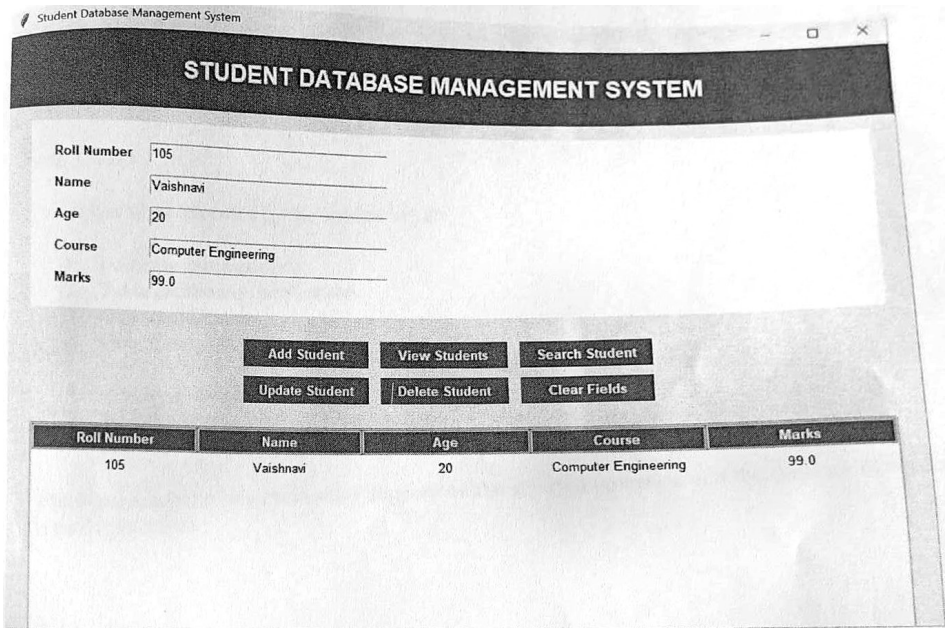


Figure 3: Searching or updating student information.

Hash Table (TABLE_SIZE = 10) after inserting the six sample records:

Bucket	**Records stored (chain)**
0	Empty
1	101 (Amit Patil) -> 111 (Rahul Deshmukh)
2	102 (Neha Joshi)
3	Empty
4	Empty
5	205 (Sneha Kulkarni) -> 215 (Pooja More)
6	Empty
7	Empty

8	308 (Kiran Shinde)
9	Empty

Roll numbers 101 and 111 (bucket 1) and 205 and 215 (bucket 5) collide, and they are stored together in a chain.

5. Description of Project

The Student Database Management System is a GUI-based application developed using Python and Tkinter. The main purpose of this project is to demonstrate the application of Hashing in a real-world student record management scenario. The system stores student information such as Roll Number, Name, Department and Marks in a hash table. The Roll Number is used as the key, and the division method (Roll Number % 10) is used as the hash function to calculate the bucket index. Collisions, which occur when two roll numbers map to the same bucket, are handled using Separate Chaining, in which every bucket holds a list of records. Because the bucket is calculated directly from the key, a student can be searched, updated or deleted by checking only the few records of one bucket instead of the entire database. Duplicate roll numbers are rejected during insertion. The Tkinter GUI makes the system interactive and easy to use, and the bucket view shows how the records are distributed inside the hash table.

6. GUI Window

The GUI provides Student Database Management System, Add Student, Search Student, Update Student, Delete Student, Display All Students and Hash Table (Bucket View).

[Paste screenshot of the GUI window (with search / add operation) here]

Figure 4: GUI Window of Student Database Management System

GUI Components

Components	**Purpose**
Main Window (Tk)	Displays the Student Database Management System application.
Label (Label)	Displays the title and field names.
Entry (Entry)	Accepts the Roll Number, Name, Department and Marks.
Button (Button)	Performs the Add, Search, Update, Delete, Display All and Clear operations.
Table (ttk.Treeview)	Displays student records along with their bucket number in rows and columns.
Text (Text)	Displays the Hash Table in bucket view with chained records.
Message Box (messagebox)	Displays success, warning and error messages.

7. Time Complexity, Space Complexity and Step Count

1.Time Complexity

Let n = number of students and m = number of buckets (m = 10). Load factor α = n/m.

Operation	**Algorithm**	**Average Case**	**Worst Case**
Insert Student	Hashing (Chaining)	O(1)	O(n)
Search Student	Hashing (Chaining)	O(1)	O(n)
Update Student	Hashing (Chaining)	O(1)	O(n)
Delete Student	Hashing (Chaining)	O(1)	O(n)
Display All / Bucket View	Traverse all buckets	O(n + m)	O(n + m)

On average each chain contains about α records, so every operation takes O(1 + α) time, which is O(1) when the load factor is small. The worst case occurs when all the records fall into the same bucket and the chain becomes a linear list of n records.

2.Space Complexity and Step Count

The student records are stored in a table of m buckets, so the storage requirement is:

O(n + m)

The program uses no extra copy of the data, so the auxiliary space for insertion, search, update and delete is O(1).
Step Count
For the six student records used in the project (n = 6, m = 10, α = 0.6):
Search using Hashing
Searching Roll No. 111 requires 1 hash calculation and 2 comparisons (101, then 111) because bucket 1 contains two records.
Searching Roll No. 102, 308 requires 1 hash calculation and 1 comparison.
Therefore, the maximum step count is approximately:
2 comparisons (the longest chain)
Linear Search (for comparison)
A linear search in the same database may need up to 6 comparisons, so hashing reduces the number of comparisons from 6 to 2.
Display All
Visits all 10 buckets and 6 records: n + m = 6 + 10 = 16 steps.

8. Conclusion

The Student Database Management System using Hashing successfully demonstrates the practical implementation of the important DSA concept of hashing in Python.
The project implements a hash table with the division method as the hash function and separate chaining for collision handling. It provides useful features such as adding, searching, updating, deleting and displaying student records, and it also shows the internal bucket structure of the hash table.
The Tkinter GUI makes the system interactive and easy to use. This project helped in understanding how hashing provides fast access to data in a real-world application such as a student database.
Thus, the project successfully combines Data Structures, Hashing and GUI programming into one practical application.

9. GitHub Repository Link:

https://github.com/rupnarvaishnavi8-code/Student-Database-Management-System

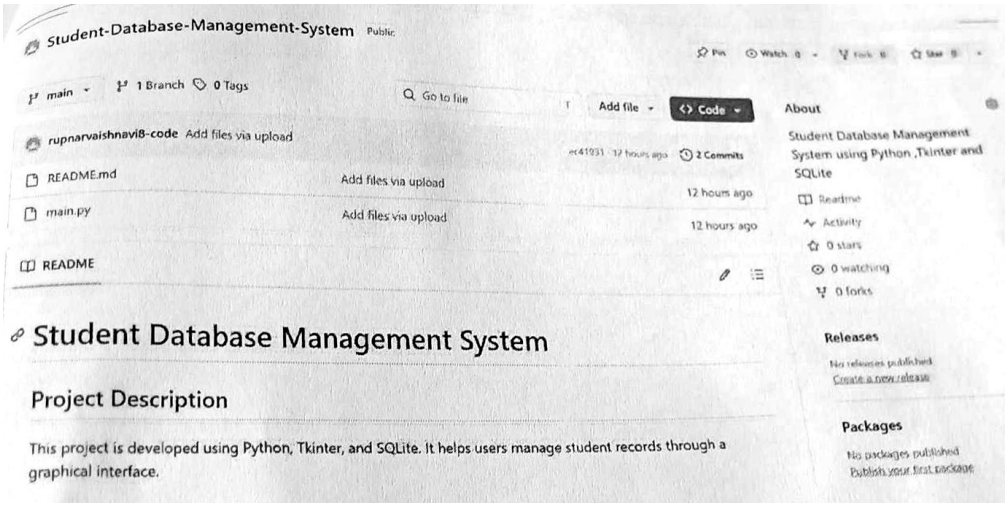


Figure 5: GitHub repository of the Student Database Management System.